

FlashTensor: Optimizing Tensor Programs by Leveraging Fine-grained Tensor Property

Runxin Zhong
zhongrx24@mails.tsinghua.edu.cn
Tsinghua University
Qingcheng.AI
Beijing, China

Yuyang Jin
jinyuyang@tsinghua.edu.cn
Tsinghua University
Beijing, China

Chen Zhang
zhang-c21@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Kinman Lei
jw-li22@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Shuangyu Li
lishuang22@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Jidong Zhai
zhaijidong@tsinghua.edu.cn
Tsinghua University
Beijing, China

Abstract

Deep neural networks (DNNs) have shown significant effectiveness in natural language processing and video applications. However, DNN models, especially for long-context tasks, introduce extremely large intermediate tensors, producing substantial memory overhead. Although considerable efforts have been made to optimize DNNs, insufficient awareness of tensor properties has hindered effective memory optimization and can lead to inefficient computations in a long-context scenario.

In this paper, we present FlashTensor[1], a DNN optimization system that reduces memory overhead and improves inference performance by leveraging fine-grained tensor properties. We first extract and identify essential tensor properties from a computation graph, such as reduce dependency and broadcastability. Then, we apply various optimizations involving transformation and kernel mapping based on these properties. Experiments on seven models demonstrate that FlashTensor achieves speedups of 1.50× and 3.24× on average for end-to-end and core module performance, respectively, compared to eight state-of-the-art works, on H100 (1.86× and 3.70× on A100).

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; • **Computing methodologies** → **Machine learning**.

[1]FlashTensor is available at https://github.com/monellz/FlashTensor

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PPoPP '25, March 1–5, 2025, Las Vegas, NV, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1443-6/25/03…$15.00
https://doi.org/10.1145/3710848.3710864

Keywords: Deep learning compiler, Long context, Memory optimization

1 Introduction

DNNs have become a transformative force across various domains, including healthcare [31], autonomous driving [19], and natural language processing [4, 17, 23, 25–27, 29, 35, 38, 39]. Their ability to learn from large datasets and solve complex problems demonstrates their versatility and effectiveness. However, this widespread adoption also brings challenges related to the increased computational demands and resource requirements of these advanced models.

There is a notable trend for DNNs towards increasing model sizes. We classify the growth of model size into two distinct categories. 1) *Parameter-determined axis (P-axis).* The P-axis primarily pertains to the hidden size of DNNs, which is being expanded to improve the quality of inference results. 2) *Input-determined axis (I-axis).* The I-axis refers to dimensions such as sequence length and image size, which are being increased to allow DNNs to process more extensive input information. We observe that these two axes exhibit *different growth rates*. For example, from Llama-65B to Llama-3.1-405B [14, 38, 39], the P-axis (hidden size) increased from 8k to 16k, with only a 2× growth factor. In contrast, the I-axis (context length) expanded from 2k to 128k, indicating a substantial growth factor of 64×, as shown in Figure 1(a).

The differing growth rates of different axes result in significant variations in the size of intermediate tensors. Figure 1(b) shows a simplified partial computation graph of attention variation [45] that includes a *MatMul* with two following *Reduce* of different reduce directions. When the sizes of the P-axis and I-axis are similar, the shapes of inputs and output to the *MatMul* operation are similar. However, with the current substantial difference between P-axis and I-axis sizes in Figure 1(b), the output of *MatMul* becomes extremely large compared with other tensors involved in the computation.

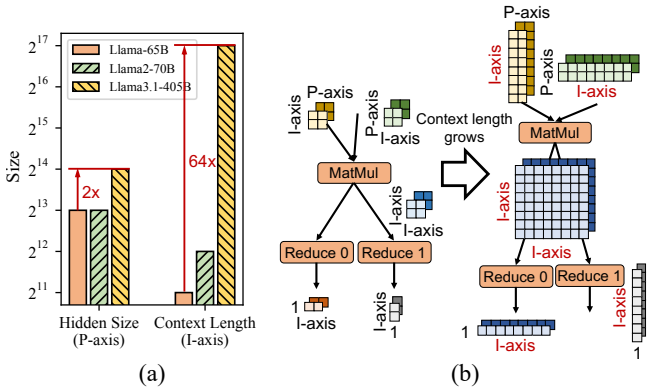


Figure 1. (a) The different growth of *Parameter-determined axis* and *Input-determined axis*. (b) A simplified example from attention variation [45].

These extremely large tensors require considerable processing time, making their efficiency critically important to the overall performance.

However, current approaches [2, 5, 15, 36, 47] fail to provide efficient solutions to these extremely large tensors as the operations on them can be very complex. As shown in Figure 1(b), the output tensor of *MatMul* is extremely large, and fusion is one of the effective ways to eliminate this tensor and reduce memory overhead. However, this tensor is subsequently utilized by two *Reduce* operators, with one performing reduction in each row and the other in each column. If fusion is applied, the complex dependencies caused by the different reduction dimensions of two *Reduce* operators will lead to low parallelism and greatly degrade performance. Being aware of properties on the intermediate tensors flowing through the computation graph edges, like the above reduces dependency on certain axes, is important to find an efficient solution to the operator graph. More types of useful properties will be discussed in Section 3.1. Though coarse-grained tensor properties like tensor size may be considered, existing deep learning compilers [21, 32, 48] overlook fine-grained tensor properties and mainly focus on operator properties, like math properties or compute resources needed to run the operators. Besides, manually developed operator libraries [10–12, 30] also fail to cover these numerous new variants of models.

In this work, we propose FlashTensor, a framework that optimizes DNN programs by leveraging fine-grained tensor properties. We observe that many tensor properties are essential for analysis and optimization to achieve optimal performance, such as *reduce dependency*, *broadcast*, *size*, and *value*. Based on this observation, FlashTensor defines some key properties of tensors and identifies them in the computation graph. Then, FlashTensor optimizes tensor programs through tensor property-aware transformations and kernel mapping strategies. Experiments on seven models show that FlashTensor achieves 1.50× end-to-end speedup and 3.24× core module speedup on average compared to state-of-the-art deep learning compilers.

In summary, we have made the following contributions:

- We conclude four essential tensor properties, and propose *Tensor Property Identifier* to systematically analyze a whole computation graph and capture the tensor properties of each tensor.
- We propose *Tensor Property-Aware Optimization* to search optimal kernels based on property-aware transformation rules and kernel mapping strategies, to achieve high computation efficiency and low memory access overhead.
- We design and implement FlashTensor, an optimizing system for tensor programs by leveraging fine-grained tensor properties, which outperforms the state-of-the-art systems.

2 Motivation

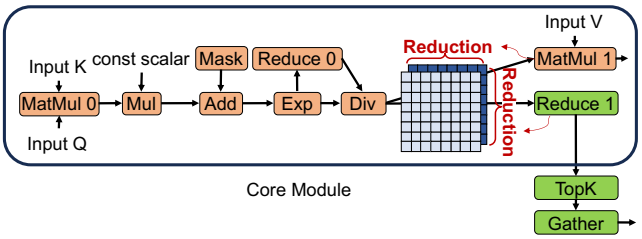


Figure 2. A Motivation example with H_2O. Green-colored operators are extra ones compared with Vanilla Attention. *Reduce 1* and *MatMul 1* apply reduction on different dimensions of the same large tensor.

One recent research trend is to use new attention variants to reduce the abundant computation of Vanilla Attention. For example, models like H_2O [45], RoCo [28], and Keyformer [1], can drop some unimportant tokens to reduce total computation volume. H_2O, a representative example shown in Figure 2, uses an additional *Reduce* operator (*Reduce 1*) after the *SoftMax* (*Exp*, *Reduce 0*, and *Div*) to compute token importance. Then a following *TopK* operator, combined with a *Gather* operator, selects and caches the most significant tokens while discarding the rest.

Inference in H_2O consists of two stages: a prefill stage and a decode stage. In the prefill stage, the input prompt is processed chunk by chunk, with important tokens selected and cached, especially when the input exceeds the cache capacity. In the decode stage, output tokens are generated sequentially, and the cache is further updated when selecting and retaining the most important tokens. Thus, the added operators affect the performance of both the pre-fill and decode stages. However, for tasks such as document summarization with long contexts, the prefill stage presents a significant

performance bottleneck. For example, in InfiniteBench [44], prompts can reach 442K tokens compared to 0.7K generated tokens—a 631× difference, resulting in a prefill-to-decode execution time ratio of 4.51 in H_2O. The primary bottleneck arises from the large tensor created during the core module of the prefill stage, as shown in Figure 2. This tensor, with a size of $O(\text{seqlen}^2)$, is produced by *MatMul 0*, incurring substantial memory access overhead. Even after optimization by TensorRT [36], the core module of H_2O accounts for approximately 57.62% of the total prefill time, yet achieving only 10.77 TFLOP/s (3.45% peak performance of A100 F16 TensorCore).

The key challenge is the two reduction operations, *MatMul 1* and *Reduce 1*, which have different reduction dimensions for the output tensor of *Div* (referred to as DivOut). To improve efficiency, each row of DivOut must be allocated to the same parallel unit for *MatMul 1*, while each column of DivOut must also reside within a single parallel unit. Consequently, there is no viable partitioning strategy for DivOut that satisfies these constraints other than utilizing a single parallel unit. As a result, current approaches such as TensorRT must handle a large tensor across multiple kernels via the slow global memory. Moreover, the two reduction operators with different reduction dimensions are the primary difference between H_2O and Vanilla Attention, making it difficult for FlashAttention [10, 11, 30] to optimize due to these structural differences. The limitation comes from their lack of awareness for fine-grained tensor properties such as reduce dependency on each dimension, making them missing fusion opportunities for *MatMul 1* with preceding operators to avoid access for large tensors.

3 System Overview

FlashTensor reduces memory overhead by utilizing fine-grained tensor properties. Before introducing our system, we first introduce the observation of key tensor properties.

3.1 Observation

Here we list some fine-grained properties and use H_2O to illustrate why they are important.

Reduce dependency. Reduce dependency refers to whether certain dimensions of a tensor are aggregated or reduced. It is important to analyze the potential parallelism opportunity of the operators. As discussed earlier, in Figure 2, the output tensor of Div has reduce dependency from two directions. *Reduce 1* forbids the inter-SM parallelization on its row dimension, and *MatMul 1* forbids the parallelization along its column dimension, making the whole computation graph difficult to parallelize. Recognizing such reduce dependencies is crucial for avoiding suboptimal parallelization patterns and searching for optimal strategies.

Broadcast. Broadcast refers to the tensor dimensions that are expanded or broadcasted to match the shape of another tensor. For example, the *Mul* operator in Figure 3 implicitly broadcasts the column dimension of its right-hand side operand. When to perform broadcast is sensitive to the total amount of computation. The amount of multiply in the example reduced from $NN'h$ to Ndh when reordered. However, such reorder does not always lead to correct results. Validating such transformations needs an in-depth analysis of the relationship between the reduce-axis and broadcast-axis, which will be further discussed in Section 4.1.

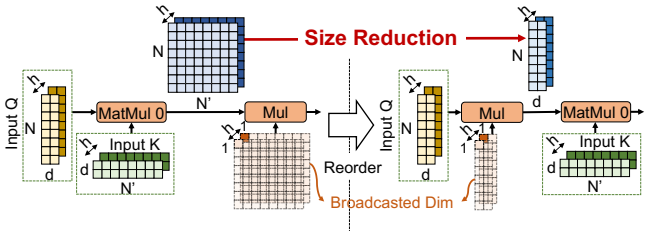


Figure 3. Broadcast example in H_2O. Reordering *MatMul 0* and *Mul* reduces intermediate tensor size.

Size. Size is an intrinsic property of a tensor, representing the total number of elements it contains. It is closely related to the memory access volume and one of the key strategies for reducing memory access overhead is to minimize it.

Value. The value property can help to reduce unnecessary computing and memory access. For example, when being aware of the triangular mask that is typically used in decoder-transformers, together with the meaning of operators on it, we can skip half of the computation of most tensors.

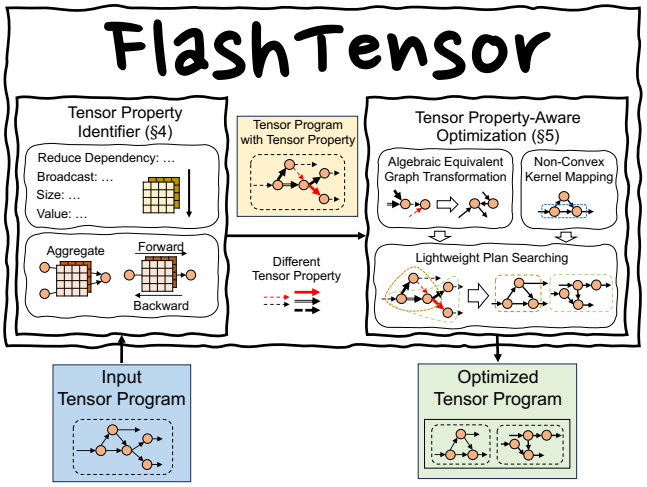


Figure 4. Overview of FlashTensor

3.2 Overview of FlashTensor

Based on the observation, we propose FlashTensor with the awareness of these fine-grained tensor properties. FlashTensor focuses on optimizing the prefill stage, which is the bottleneck for long-context tasks, as discussed in the Motivation section. The overview of FlashTensor is shown in Figure 4, which consists of two main modules: *Tensor Property Identifier* and *Tensor Property-Aware Optimization*. (1) Firstly, the *Tensor Property Identifier* module takes a tensor program as input, represented as a computation graph where nodes denote operators and edges represent tensors, and captures all the fine-grained properties of each tensor within the computation graph, including reduce dependency, broadcast, size, and value (Section 4). (2) Then, the *Tensor Property-Aware Optimization* module searches optimal plans based on the graph transformation rules and kernel mapping strategies. The graph transformation rules are designed to reduce the intermediate tensor size with specific restrictions, and the kernel mapping strategies target high-efficiency candidate kernels by considering memory access and computation intensity, as well as parallelism. The optimized tensor program is finally generated, which is ready for execution as efficient code (Section 5).

4 Tensor Property Identifier

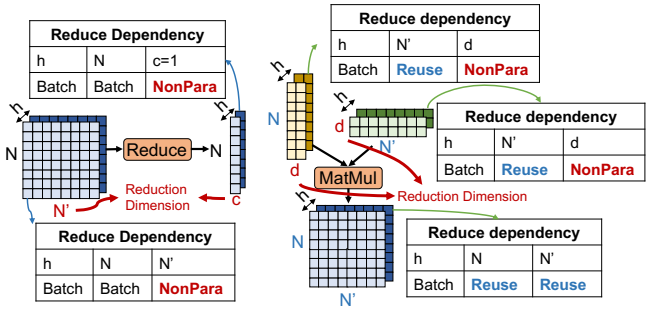
In this section, we first formally define the properties we observe that are essential for subsequent analysis and optimization, then introduce how FlashTensor identifies these properties in computation graphs.

Table 1. Tensor properties in FlashTensor. The second column lists the potential values for each corresponding property.

Per-Dimension Property		
Reduce dependency	NonPara	Non-parallelizable dim.
	Batch	Embarrassingly parallelizable dim.
	Reuse	Parallelizable dim with data reuse.
Broadcast	No	Unbroadcasted dim.
	Yes	Broadcasted dim.
Entire Tensor Property		
Size	An integer	The total size of the tensor.
Value	Var	The tensor value is undetermined.
	Zero	The tensor is all zero.
	PosConst	The tensor is a positive constant.
	NegConst	The tensor is a negative constant.
	PosInf	The tensor is positive infinity.
	NegInf	The tensor is negative infinity.
	NaN	The tensor is not a number.

4.1 Property Definition

As presented in Table 1, the tensor properties are classified into two distinct categories: 1) *Per-Dimension Property* focuses on attributes specific to individual dimensions of the tensor, providing insights into aspects such as reduce dependency and broadcastability. 2) *Entire Tensor Property*, encompasses attributes that describe the tensor as a whole, such as its overall size and possible constant value.



(a) Dependency for Reduce (b) Dependency for Batched MatMul

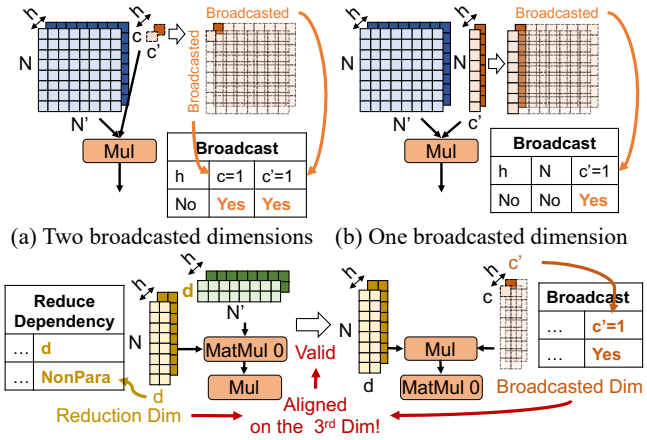
Figure 5. Reduce dependency property

Reduce Dependency. The reduce dependency property is a multi-valued attribute that describes the inter-dependence between tensor dimensions and reduction operations, which is crucial for optimizing computation efficiency. It has three values: 1) *NonPara* indicates dimensions that cannot be partitioned for parallel execution due to data dependencies from reduction operations. These dimensions require processing by a single unit, preventing parallelization. For instance, in Figure 5(a), the reduction dimensions of both input and output are classified as *NonPara* as they necessitate data aggregation on the entire dimension. 2) *Reuse* refers to dimensions that can be parallelized while enabling data reuse through tiling. For example, in Figure 5(b), the row dimensions in *MatMul* can be parallelized, with tiling improving memory efficiency by reusing the right-hand side operand. This approach also applies to broadcast-supporting operations like *Add* and *Mul*, where tiling followed by parallelization balances computational efficiency and memory access. 3) *Batch* represents dimensions that can be fully parallelized without data reuse, such as the batch dimension in Batched Matrix Multiplication shown in Figure 5(b), and the dimensions except reduction dimension for *Reduce* in Figure 5(a).

Broadcast. It is an attribute indicating whether a tensor dimension will be broadcast by a subsequent operator. A dimension is considered *Broadcasted* if the tensor is an operand of a broadcasting-supported operator at that dimension. Figure 6(a)(b) shows examples of how dimensions are expanded to meet operator shape requirements.

Size. It represents the total element count in a tensor, calculated as the product of all dimensions, as shown in Figure 7.

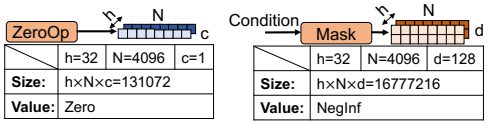
Value. It indicates if a tensor maintains a constant value during computations, as illustrated in Figure 7. Unlike traditional compilers that treat constants as scalars, FlashTensor



(a) Two broadcasted dimensions (b) One broadcasted dimension

(c) Reorder MatMul 0 and Mul based on alignment of broadcast and reduce dependency on the same dimension

Figure 6. Broadcast property



(a) Zero-valued tensor produced by ZeroOp

(b) Mask may produce NegInf in some Condition

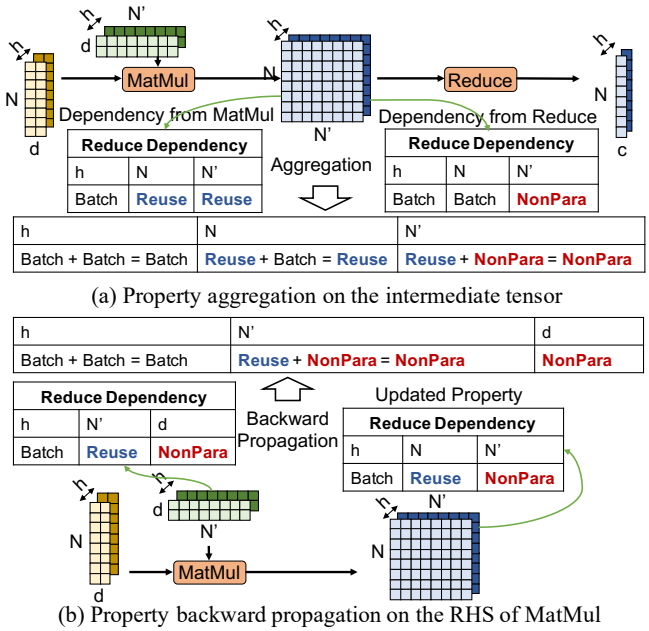
Figure 7. Size and value property

instead handles it at the tensor level, enabling a broader range of optimization opportunities. Tensors with constant values can be pre-computed or efficiently stored, minimizing dynamic updates during execution. In FlashTensor, constant value information is represented as an enumeration rather than the actual value, balancing information effectiveness and overhead.

4.2 Dataflow-Based Property Identification

Some basic properties, such as size, can be easily extracted from tensors. However, complex properties like reduce dependency require sophisticated analysis for accurate annotation. For instance, in an individual operator, the reduce dependency of input and output tensors can be naively annotated based on their computational semantics, even without fully implementing the operator, as illustrated in Figure 5. However, in a computation graph with multiple operators, the reduce dependency property of a tensor is influenced by both subsequent and preceding operators, making it challenging to determine its definitive value.

To tackle this issue, we propose a two-stage dataflow-based property identification algorithm. 1) *Property Propagation*: Tensor properties are propagated both forward and backward based on the computational semantics of each operator. 2) *Property Aggregation*: Properties propagated from various operators are aggregated for each tensor to determine the final property value. The algorithm leverages the inherent monotonic priority (for example, reduce dependency types: NonPara > Reuse > Batch). When different types converge on the same tensor dimension, the higher priority value is retained, ensuring accurate property representation across the entire graph. These stages are executed iteratively until the properties stabilize.



(a) Property aggregation on the intermediate tensor

(b) Property backward propagation on the RHS of MatMul

Figure 8. Example for reduce dependency identification. Forward propagation is similar to the backward one. + means the operation that produces value with higher priority.

Figure 8 provides examples of reduce dependence identification for further illustration. Figure 8(b) illustrates how the output of *MatMul* updates the reduce dependency of the right-hand side operand from Reuse to NonPara at dimension N' through backward propagation. Figure 8(a) shows how properties are aggregated between *MatMul* and *Reduce*, where the middle tensor receives differing reduce dependency values from both operators.

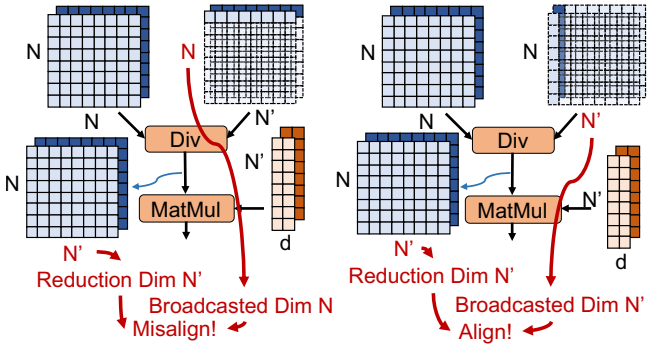
5 Tensor Property-Aware Optimization

FlashTensor further optimizes tensor programs based on the identified tensor properties. The optimizations are based on two rules, *algebraic equivalent graph transformation* and *non-convex kernel mapping*, as well as a *lightweight plan searching* method. The *algebraic equivalent graph transformation* provides a series of transformation rules, allowing for intermediate tensor size changes with equivalent transformations, and the *non-convex kernel mapping* proposes a new kind of candidate kernel for efficient execution. The *lightweight plan searching* achieves efficient plan generation

by property-constrained pruning and low-cost performance models.

5.1 Algebraic Equivalent Graph Transformation

The size of intermediate tensors dominates the memory access overhead. Therefore, we propose a transformation method, which mainly focuses on the size change of intermediate tensors by performing algebraic equivalent transformations on computation graphs. It enables the subsequent step of searching for the computation graphs with the smallest intermediate tensor size across the entire search space, thereby reducing memory overhead. *Broadcast is the essence of bringing about changes in intermediate tensor sizes.*



(a) Invalid reorder (b) Valid reorder

Figure 9. *MatMul* and *Div* with conditioned validity for reorder

Table 2. Broadcast property-based transformation rules. A, B and C are tensors. +, $\odot$ and / are element-wise add, mul and div. AB means matrix multiplication for A and B. All the operators except matrix multiplication support broadcast.

Rules	Comment	Conditioned
$A+B=B+A$	Commutativity	No
$A\odot B=B\odot A$	Commutativity	No
$A+(B+C)=(A+B)+C$	Associativity	No
$A\odot(B\odot C)=(A\odot B)\odot C$	Associativity	No
$\sum_i(A+B)=\sum_i A+\sum_i B$	Associativity	No
$A\odot C+B\odot C=(A+B)\odot C$	Distributivity	No
$A/C+B/C=(A+B)/C$	Distributivity	No
$\sum_i(A\odot B)=(\sum_i A)\odot(\sum_i B)$	Distributivity	Yes
If A or B is broadcasted on dimension i for $A\odot B$		
$\sum_i(A/B)=(\sum_i A)/(\sum_i B)$	Distributivity	Yes
If B is broadcasted on dimension i for A/B		
$(A\odot B)C=(AC)\odot B$	Distributivity	Yes
If B is broadcasted on reduction dimension for AC		
$(A/B)C=(AC)/B$	Distributivity	Yes
If B is broadcasted on reduction dimension for AC		

Broadcast property-aware transformation rules. Consequently, we propose a series of transformation rules that not only consider tensor sizes but also take into account the broadcast property along each dimension. Specifically, Table 2 lists all the transformation rules, which can be divided into two main parts: *1) Always valid transformations.* These transformations are always valid whatever the broadcast property of their operands is. Their validity is guaranteed by the three mathematical laws, including commutativity, associativity, and distributivity for different operators and their combinations. *2) Broadcast property-constrained transformations.* The validity of certain transformations depended on the broadcast. In these transformations, broadcast is treated as an essential property to ensure correctness. For example, as illustrated in Figure 9(a), in a scenario where the RHS of *Div* broadcasted on dimensions N, the reorder is invalid because the reduction in *MatMul* would disrupt computation semantics and lead to incorrect results. However, as shown in Figure 9(b), if the right-hand side operand of *Div* is broadcasted along a dimension that aligns with the reduction dimension of *MatMul*, the reorder becomes valid.

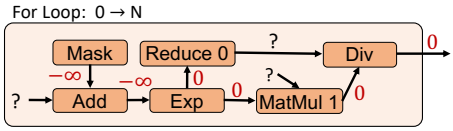


Figure 10. Value example

Value property-aware transformation rules. We leverage the value property to optimize loop iterations by eliminating unnecessary iterations. Specifically, it focuses on skipping certain iterations in a For loop without affecting the final output. As illustrated in Figure 10, some iterations may produce constant values, like -∞, from the *Mask* operator. Because these constant values propagate unchanged through the For loop, we can identify them and safely skip those iterations. This not only maintains the correctness of the output but also reduces computations.

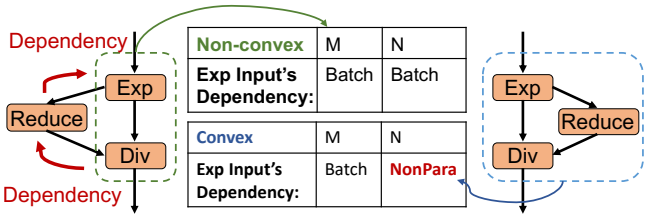
5.2 Non-Convex Kernel Mapping

Kernel mapping involves assigning operators in a computation graph to GPU kernels for execution on modern hardware platforms [15]. For a given computation graph, naively fusing all operators into a single kernel often results in suboptimal performance due to complex reduce dependency and limited parallelism. It is essential to identify proper kernels to achieve high efficiency.

We first give the formal definition of the kernel in the state-of-the-art works [15].

Definition 1 (Kernel). *For a computation graph $G=(V,E)$, a set of nodes $K\subseteq V$ forms a Kernel if there do not exist nodes $p_1,p_2\in K$ and another node $q\in V\backslash K$ such that $p_1\overset{G}{\rightsquigarrow}q$ and $q\overset{G}{\rightsquigarrow}p_2$, where $x\overset{G}{\rightsquigarrow}y$ indicates that there exists a path from x to y in G.*

This definition takes a *Convex* subset of operators within a computation graph as a kernel. Convex implies that such kind of kernel cannot include operators that depend on itself via an external operator, as shown in Figure 11(a). Korch argues that such a kernel cannot be executed due to circular dependencies [15] (i.e. *Div* depends on *Reduce* which depends on *Exp*). However, such circular dependency can be resolved through other kernels. Specifically, it is not necessary for a kernel to generate its inputs; it suffices if another kernel provides these inputs. For example, to resolve the circle dependency in Figure 11(a), another kernel containing *Exp* and *Reduce* can provide the output of *Reduce*.



(a) A non-convex kernel with circular dependency

(b) A convex kernel with stricter reduce dependency

Figure 11. Examples for non-convex and convex kernels

By relaxing the convex requirement, i.e. supporting kernels that is *Non-Convex*, a kernel does not require including any operator on the path of internal operators, bringing the following two benefits: 1) Less computation: Compared with a convex kernel, a non-convex kernel is able to have fewer operators due to the relaxation of requirements, directly avoiding corresponding computation. 2) Potential relaxed reduce dependency: Excluding some operators can potentially make the subgraph's dependency simpler. As depicted on Figure 11(a) and (b), if we exclude Reduce to form a non-convex kernel containing Exp and Div, its reduce dependency is much simpler because there are only element-wise operators remaining in such kernel. Below we present a formal definition of a non-convex kernel as considered within FlashTensor.

Definition 2 (Non-Convex Kernel). *For a computation graph $G = (V, E)$, a set of nodes $K \subseteq V$ forms a Non-Convex Kernel if there* ***exist*** *nodes $p_1, p_2 \in K$ and another node $q \in V \backslash K$ such that $p_1 \overset{G}{\rightsquigarrow} q$ and $q \overset{G}{\rightsquigarrow} p_2$.*

Given this definition, we can discuss the input and output tensors of a kernel. 1) *Kernel Input.* The input tensors for a kernel are determined based on the dependencies of all operators within the kernel. Specifically, a tensor is classified as an input to the kernel if it is not the output of any operator within the kernel. 2) *Kernel Output.* In contrast, identifying the output tensors of a kernel is not as straightforward as deducing inputs. Output tensors cannot be determined solely by examining data dependencies, as intermediate tensors within the kernel could potentially be designated as outputs to resolve the circular dependency of other kernels. Therefore, it is necessary to *explicitly* specify which intermediate tensors should be considered as outputs for the kernel.

Algorithm 1: Kernel Identification

```
Data: Computation Graph G, Arithmetic Intensity threshold H
Result: Candidate Kernels K in G
Function Search(G):
    K = ∅;
    max_para_units ← query underlying accelerators;
    foreach connected subset X ∈ G do
        AI ← compute arithmetic intensity of X;
        num_para_units ← get parallelism of X;
        if AI ≥ H and num_para_units ≥ max_para_units
          then
            K = K ∪ {X};
    return K;
```

5.3 Lightweight Plan Searching

Based on the proposed rules, the next step is to efficiently search for a high-performance plan in a vast search space. We mainly adopt a two-stage search on the transformation and the kernel mapping stage. Below we introduce the fast search methods of each stage, respectively.

5.3.1 Stage 1: Transformation. The search goal of this stage is to minimize the intermediate tensor size as much as possible to reduce memory access overhead. However, solving this search problem is challenging due to the search space. A straightforward greedy search method is insufficient as it tends to converge to local optima. For example, some transformations may not immediately affect the size of input and output tensors but can pave the way for substantial future reductions in tensor sizes. This limitation causes the greedy algorithm to miss the global optimum.

Tensor size-minimized search. To address this challenge, we propose a tensor size-minimized search algorithm based on Simulated Annealing [3], which is a probabilistic algorithm designed to find an approximate global minimum with acceptable costs. We explore various plans of transformations probabilistically, occasionally accepting large or same-sized tensors to escape local optima and gradually reducing the probability of such acceptance as the algorithm progresses. Although this algorithm does not guarantee an optimal plan, it offers control over execution time, making it feasible for practical use.

5.3.2 Stage 2: Kernel Mapping. Due to the introduction of non-convex kernels in FlashTensor, the search space for kernels has increased further. We use a roofline-based performance model to predict the performance of each plan for finding the optimal plan from the pruned space. To quickly

identify high-performance kernels, we also perform significant pruning to eliminate candidates with poor performance based on property constraints.

Roofline-based performance model. To identify all the candidates including convex kernels and non-convex kernels, we employ a Brute Force Search approach to explore all possible connected operator subsets within a computation graph. Note that a single connected subset can correspond to multiple candidates, depending on whether intermediate tensors within the subset are marked as output or not. To predict the performance of candidates, we propose a roofline-based performance model, which mainly considers two metrics:
1) Arithmetic Intensity. It is commonly used in roofline analysis [42] for evaluating a single operator, defined as the ratio of computation volume to memory access volume. For a candidate containing multiple operators, we extend it as follows:

$$AI = \frac{\sum_{op \in \text{Kernel}} Comp(op)}{\sum_{t \in \text{In(Kernel)} \cup \text{Out(Kernel)}} Mem(t)}$$

Here, $Comp(op)$ represents the computation volume of an operator op within the kernel, and $Mem(t)$ represents the size of tensor t that are inputs or outputs of the kernel.
2) Parallelism. This metric quantifies the number of parallel execution units available for kernel parallelization, derived from the reduce dependency property identified from the input tensors. For the Batch dimension, all sizes contribute to parallelism. For the Reuse dimension, we use a heuristic to determine the optimal parallelism based on dimension size. Specifically, if the dimension size is too small, efficient data reuse through tiling is not feasible, and all sizes are counted into the overall parallelism. Conversely, with a large dimension size, we apply a tiling strategy with a pre-defined block size, and the tile count contributes to the parallelism.

Property-constrained pruning. Given the extremely large number of candidates, a property-constrained pruning strategy is proposed to reduce space and focus on promising candidates. During the iteration, kernels are pruned if their parallelism is less than the available parallel unit count, or their arithmetic intensity below a pre-defined threshold. It helps to eliminate inefficient kernels that do not meet performance expectations and improve search efficiency.

The entire search algorithm is shown in Algorithm 1. It begins by determining available parallel execution unit count, such as GPU Streaming Multiprocessors (SMs). Then we iterate over connected subsets of the computation graph, pruning kernels that fall below performance thresholds for parallelism or arithmetic intensity. Once candidate kernels are pruned, we follow the prior work [15] to formalize the optimal candidate search as a binary linear programming (BLP) task and solve it.

6 Evaluation

FlashTensor is implemented in 10K lines of C++ and 2K lines of Python code, based on MLIR [18] and Triton [37]. FlashTensor accepts an ONNX-format tensor program that can be easily exported by existing tools from native TensorFlow/PyTorch programs, and translates it into a valid MLIR code. We implement two Dialects in MLIR, FT and FTTriton. FT defines tensor operators such as element-wise operators, reduce, reshape, and matrix multiplication and corresponding transformations implemented by MLIR passes. FTTriton serves as a bridge from FT to Triton DSL, defining operators that are similar to Triton. After applying all optimizations, the final MLIR codes will be converted to a valid Triton DSL.

6.1 Evaluation Setup

Platforms. We evaluate FlashTensor on two platforms: 1) one NVIDIA A100-PCIE-40GB GPU with two AMD EPYC 7742 64-Core CPUs, and 2) one H100-PCIE-80GB with AMD EPYC 7453 28-Core CPUs. The software versions are Python 3.10, CUDA 12.1, and GCC 12.2.

Workloads. We use seven models that support inputs with long sequences to evaluate FlashTensor, including H_2O [45], RoCo [28], Keyformer [1], SnapKV [20], Corm [9], Vanilla Attention [40] (V.A.), Gemma2 [34]. The brief descriptions of these models are shown in Table 3. For the end-to-end experiments, all models use *Llama-2-7b* [39] as the base model with FP16 weights, head num of 32, and head dim of 128, where the only difference is their attention module for fair comparison.

Table 3. Basic information of evaluated models.

Models	Description
H_2O	It utilizes accumulated attention score to drop heavy hitters tokens that contribute little to the final attention score.
RoCo	It notices the relation between the standard deviation of the attention probabilities and the importance of tokens and exploits it to guide token eviction.
Keyformer	It utilizes a Gumbel softmax-based score function for key token identification.
SnapKV	It exploits clustering to retain attention features for important tokens.
Corm	It defines a long-term minor key and drops those tokens that are regarded as a long-term minor key.
V.A.	It is the standard attention that is widely used in current transformer-based models.
Gemma2	It is a newly proposed model that introduces logit soft-capping for attention layers.

Baselines. Seven state-of-the-art (SOTA) deep learning compilers and two operator libraries are used for comparison. 1) *Deep learning compilers*: PyTorch 2.2.2 [24] and corresponding TorchInductor [2], TensorRT 10.0.1 [36], TVM

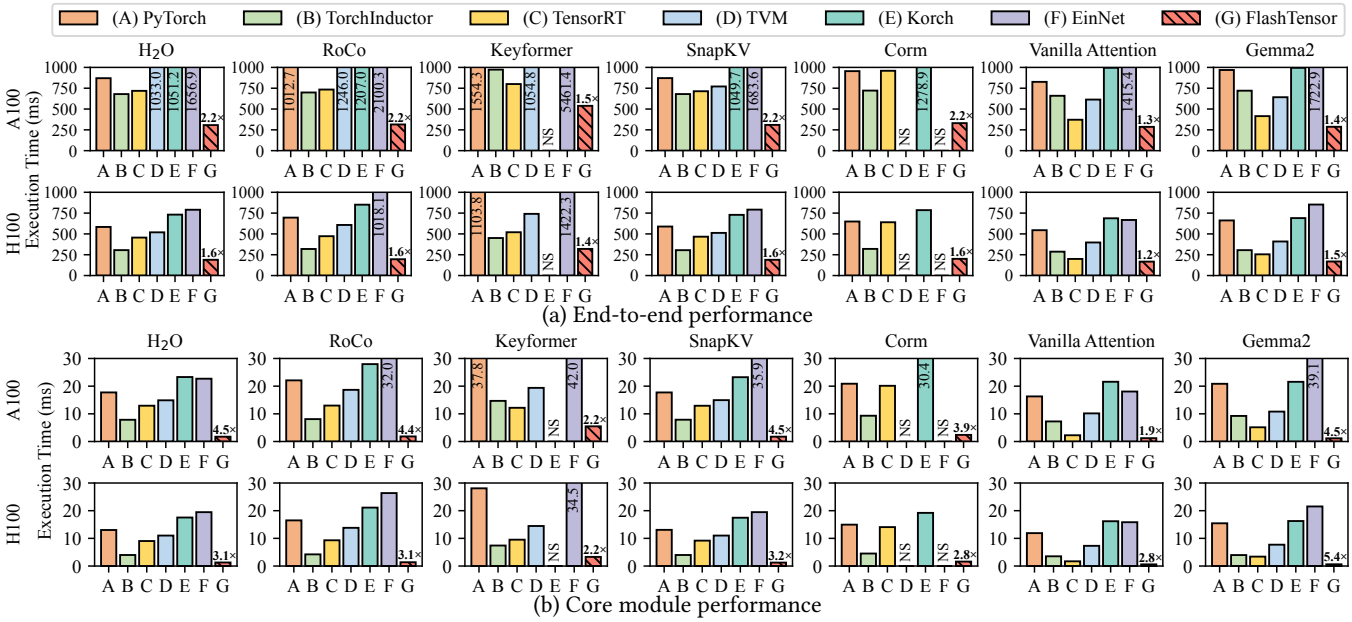


Figure 12. End-to-end and core module performance comparison of seven evaluated models with SOTA systems on A100 and H100 GPU. NS means NotSupport. Bars for baselines of too long running time are truncated, and their execution times are marked on the bars. The numbers above FlashTensor's bars show FlashTensor's speedups over the best baseline.

0.16.0 [5] with MetaScheduler [6, 46], Korch [15], and EinNet [47]. 2) *Operator libraries*: FlashAttention 2.6.2 [10, 11] and FlashInfer 0.1.0 [12].

6.2 End-to-End and Core Module Performance

We first compare the end-to-end inference performance of models of FlashTensor with SOTA deep learning compilers. As shown in Figure 12(a), FlashTensor outperforms the best baseline by up to 2.22× speedup on A100 and 1.62× speedup on H100. The performance gain of FlashTensor mainly comes from the highly optimized core modules. Figure 12(b) shows the performance of core modules, such as attention variations. FlashTensor outperforms the best baseline by up to 4.52× speedup on A100 and 5.43× speedup on H100.

We observed that some models are structurally similar. The performance of these models is roughly equivalent with FlashTensor. However, after being optimized by SOTA deep learning compilers, their performance can vary significantly. For example, as shown in Figure 14(a), Gemma2 only introduces minor modifications to Vanilla Attention. With TensorRT, the inference times for Gemma2 and V.A. on the A100 are 5.13 ms and 2.25 ms, respectively. The significant performance gap arises because TensorRT has a specific pre-written rule (FlashAttention-like) tailored to the attention structure in V.A., which fails to match the minor modified structure in Gemma2. However, with FlashTensor, the inference times for Gemma2 and VA on the A100 are 1.14 ms and 1.20 ms, respectively, demonstrating comparable performance that surpasses TensorRT. It is because FlashTensor automatically searches for the optimal plan through tensor property-aware rules, which can adapt different models while employing value-aware loop iteration elimination techniques to further reduce unnecessary computation caused by the causal mask.

6.3 Scalability for Different Sequence Lengths

Figure 13 shows the scalability under different sequence lengths on H_2O in the aspects of computational efficiency and memory occupancy.

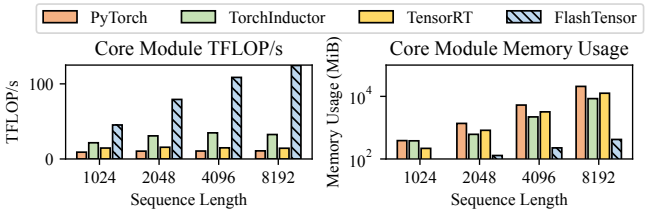


Figure 13. Memory footprint and TFLOP/s for different sequence length in H_2O on H100. Batch size is 1.

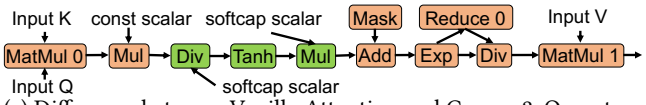
Computational efficiency. Floating points per second (FLOP/s) is an important indicator to measure kernel computational efficiency or throughput. FlashTensor achieves higher FLOP/s when sequence length becomes longer, due to its lower memory access overhead. TorchInductor and TensorRT share the same kernel mapping strategies, requiring reading and writing intermediate tensors with $O(\text{seqlen}^2)$

size from and to global memory in their cuda kernels, thus incurring high memory overhead.

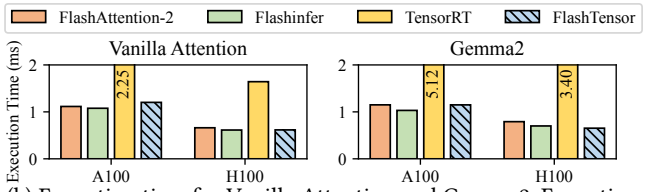
Memory footprint. We further use the memory footprint of kernels to demonstrate the intermediate tensor size reduction brought by FlashTensor. Compared to PyTorch and TensorRT, FlashTensor exhibits a significantly smaller memory footprint in global memory, especially as the sequence length increases. The primary reason for this difference is that PyTorch and TensorRT allocate large intermediate tensors on slow global memory that scale quadratically with the sequence length, $O(\text{seqlen}^2)$. In contrast, FlashTensor aims to minimize the allocation of such large tensors, thereby maintaining a more efficient and stable memory usage as sequence lengths grow.

6.4 Comparison with Operator Libraries

We further compare FlashTensor and TensorRT with hand-tuned operator libraries, FlashAttention and FlashInfer, which both only support Vanilla Attention and Gemma2.



(a) Difference between Vanilla Attention and Gemma2. Operators with green color are the extra structure of Gemma2.



(b) Execution time for Vanilla Attention and Gemma2. Execution times (ms) are marked on the truncated bars.

Figure 14. Comparision with operator libraries

Gemma2 [34] introduces logic soft capping to Vanilla Attention, as illustrated in Figure 14(a). In Figure 14(b), FlashTensor achieves comparable performance with FlashInfer and FlashAttention, both of which employ hand-tuned operators designed with domain expertise and human efforts. Although FlashTensor does not surpass these operator libraries, it offers the advantage of generalization and optimization across various scenarios without manual tuning.

6.5 Breakdown Analysis

We conduct a breakdown analysis to demonstrate the performance effects of each component for FlashTensor, as shown in Figure 15.

The baseline is PyTorch with a speedup of 1. FlashTensor first applies operator fission to decompose complex operators into basic ones, such as converting *Softmax* to *Exp*, *Reduce*, and *Div*. It introduces smaller operators with increased memory access volume, reducing the overall performance.

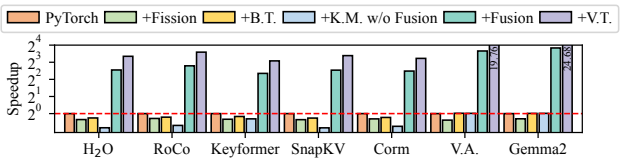


Figure 15. Core module performance breakdown on H100. Fission is splitting complex operators into basic ones. B.T. means broadcast property-aware transformation. K.M. w/o Fusion means kernel mapping without fusion. V.T. means value property-aware transformation.

Next, the broadcast property-aware transformation (B.T.) is proposed to minimize the size of intermediate tensors. This not only enhances performance but also enables future fusion by meeting hardware constraints. Kernel mapping (K.M. w/o Fusion) is then employed to identify convex and non-convex kernels with high parallelism efficiency. The identified kernels recompute some operators to resolve the circle dependency of non-convex kernels, thereby still reducing performance without fusion. Fusion delivers the most significant performance improvement by reducing memory access overhead and leveraging the high parallelism efficiency of identified kernels, achieving up to a 14.2× speedup, as illustrated in Figure 15. Finally, the value property-aware transform (V.T.) exploits the causal mask to eliminate redundant computation, further achieving an additional near 2× speedup across all models.

6.5.1 Per-stage analysis.

We further conduct a per-stage analysis to reveal the performance impact of transformation and kernel identification rules, respectively.

Transformation stage. FlashTensor proposes a series of broadcast property-aware transformation rules to minimize the size of intermediate tensors. As shown in Table 4, the total size of intermediate tensors is significantly decreased by about 3 GiB, making less memory usage and lower memory access overhead consistent with Figure 13. Besides, the search costs for the transformation stage are very small, around hundreds of milliseconds.

Table 4. Changed size and execution time of broadcast-based transform under sequence length of 4096

Cases	Intermediate tensor size change	Time (ms)
H_2O	-3.30 GiB	169.1
RoCo	-3.18 GiB	161.57
Keyformer	-3.28 GiB	223.69
SnapKV	-3.30 GiB	172.2
Corm	-3.30 GiB	166.5
Vanilla Attention	-3.98 GiB	187.25
Gemma2	-3.39 GiB	238.25

Table 5. Search performance. #Op represents the number of operators in core modules. #Candidate represents the number of candidates after pruning and the number of possible candidates is in the brackets.

Cases	#Op	#Candidate (#Possible)	End-to-end Search Time		
			FlashTensor	TVM	Korch
H_2O	16	5 (297K)	3.03 sec	1.8h	1.6h
RoCo	19	7 (2.6M)	8.40 sec	2.3h	3.5h
Keyformer	24	10 (169M)	251.20 sec	3.3h	Failed
SnapKV	17	5 (207K)	2.97 sec	2.1h	2.1h
Corm	17	5 (480K)	4.14 sec	Failed	3.0h
V.A.	15	4 (70K)	2.42 sec	1.5h	1.4h
Gemma2	19	4 (1.7M)	10.61 sec	1.6h	2.6h

Kernel mapping stage. FlashTensor faces a large search space due to further introducing non-convex kernel definition when determining the optimal kernel mapping strategy. As illustrated in Table 5, even with a small number of operators, the search space remains vast. For instance, the attention module of Keyformer only has 24 operators, yet the size of the whole search space is 169M.

As shown in Table 5, Korch and TVM experience long tuning times, at least about 2 hours, but still find suboptimal strategies. The long tuning time primarily comes from profiling every identified candidate kernel, requiring substantial time to discover effective schedules. However, although FlashTensor has many more candidate kernels than Korch, we consider tensor properties and effectively prune candidate kernels that are likely with poor performance, significantly reducing the candidate set, which enables quick optimal kernel selection at the second level.

6.6 Case Study: H_2O

We use H_2O as a case to understand how the model is optimized by leveraging several essential properties in FlashTensor. This experiment is conducted on A100 GPU.

Table 6. Performance metrics for H_2O on A100. The DRAM and L2 columns show the amount of memory access by read.

System	Name	Time (ms)	DRAM (MB)	L2 (MB)
TensorRT	K0	1.05	75.19	2150
	K1	3.43	2280	4270
	K2	7.03	3620	172300
	K3	0.85	1110	1610
FlashTensor	K4	0.89	100.70	1140
	K5	0.70	67.86	594.79

Reduce dependency and size properties. In FlashTensor, non-convex kernel identification leverage reduce dependency and size properties to minimize memory access overhead while maximizing parallelism. As shown in Figure 16(a) and Figure 16(b), FlashTensor utilizes only 2 kernels for H_2O, compared to TensorRT's 4 kernels, achieving a speedup of 7.7×. While TensorRT's convex kernels ensure high parallelism, they require large intermediate tensors, resulting in significant DRAM access (several GBs for kernels K1, K2, K3). In contrast, the *Non-Convex* kernel K5 identified by FlashTensor resolves a circular dependency involving the *Reduce 0* operator, allowing for the fusion of *MatMul 0* and *Reduce 1* without sacrificing parallelism efficiency. The reduce dependency properties of inputs Q and K for K5 indicate parallelization opportunities on the N dimension of K (Reuse), achieving high parallelism with only $O(\text{seqlen})$ size for input and output tensors, as evidenced by the reduced DRAM access reported in Table 6.

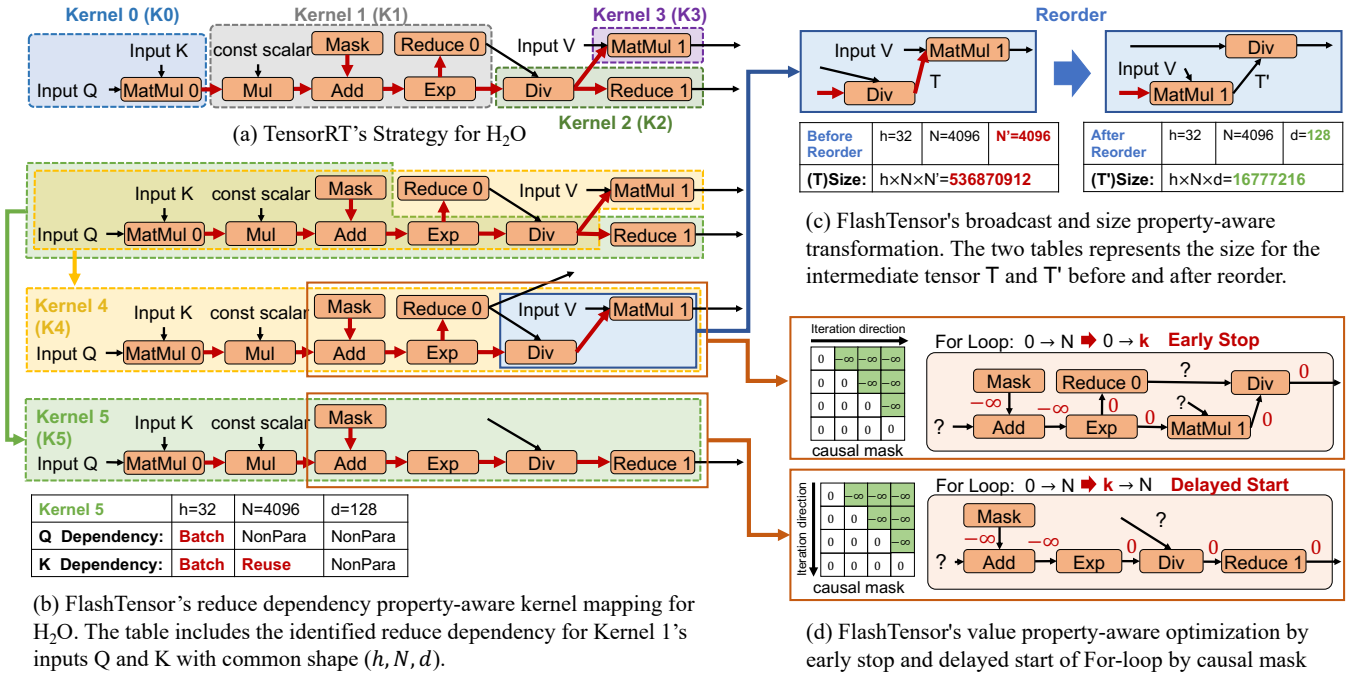
Broadcast and size properties. FlashTensor also incorporates broadcast and size properties to minimize intermediate tensor sizes. In K4, FlashTensor reorders *Div* and *MatMul 1* (in Figure 16(c)), reducing the intermediate tensor size. This reorder is valid due to the alignment between the broadcasted dimension of *Div* and the reduce dimension of *MatMul 1*. With this optimization, the intermediate tensor size is reduced by 96.86%.

Value property. Additionally, FlashTensor utilizes the value property to eliminate unnecessary computations through early stopping and delayed starting of iterations. The optimization opportunities detected by FlashTensor, as illustrated in Figure 16(d), arise from the causal mask introducing -∞ in the upper triangular portion, allowing FlashTensor to determine that certain iterations do not impact final results and eliminate nearly half of the computations.

7 Related Work

Compute graph optimization. TensorRT [36] and XLA [43] optimize compute graphs by transforming them with predefined rules. TASO [16], PET [41], and EinNet [47] can search new transformation rules. Korch splits the operators into fine-grained operators and searches the best combination of them. However, without the aware of tensor properties, they can only generate simple-structure custom operators like eOperator in EinNet and leave other operators to be optimized by hand-tuned libraries, missing opportunities to fuse complex operators such as matrix multiplication.

Property-aware code generation. Fusion-based works including Astitch [48], DNNFusion [21], SmartMem [22] and Welder [32] enable fusion by considering the relationship between their input and output elements of each operator. However, without leveraging fine-grained tensor properties on every dimension, these methods are limited to well-aligned workloads and have suboptimal performance with complex cases, such as operations involving multiple reduction directions. For example, DNNFusion [21] and SmartMem [22] categorize operations like *GEMM* and *Reduce* as highly complex (e.g., "Many-to-Many" or "Input Layout Dependent & Variable") and avoid fusing them, assuming that such fusion could have potential negative impacts on register and



(a) TensorRT's Strategy for H_2O

Kernel 5	h=32	N=4096	d=128
Q Dependency:	Batch	NonPara	NonPara
K Dependency:	Batch	Reuse	NonPara

(b) FlashTensor's reduce dependency property-aware kernel mapping for H_2O. The table includes the identified reduce dependency for Kernel 1's inputs Q and K with common shape (h, N, d).

(c) FlashTensor's broadcast and size property-aware transformation. The two tables represents the size for the intermediate tensor T and T' before and after reorder.

(d) FlashTensor's value property-aware optimization by early stop and delayed start of For-loop by causal mask

Figure 16. Case study on H_2O

cache efficiency. However, in long-context scenarios, such fusion can be both feasible and beneficial if input tensor axes are sufficiently long to provide parallelism, and if data dependencies are carefully managed to avoid parallelization along reduction dimensions. This requires an analysis of fine-grained tensor properties, such as size and reduction dependencies, as proposed by FlashTensor.

Operator-level code generation. cuDNN [7] and cuBLAS [8] provide efficient implementations of traditional operators like *GEMM* and *Convolution*. FlashAttention [10, 11, 30] and Flashinfer [12] provides fast kernels for attention, fusing all operators into a single kernel to minimize memory access overhead, which is a specific case covered by FlashTensor. However, for more complex scenarios like H_2O, simply fusing all operators, as in FlashAttention-like methods, results in suboptimal performance due to poor parallelism and hardware underutilization caused by data dependencies. FlashTensor overcomes these limitations by employing non-convex kernel mapping and recomputation, achieving both high parallelism and reduced memory access overhead through its compiler-driven approach. For domain-specific languages like TVM [5], Triton [37], FreeTensor [33], and FlexAttention [13], they enable custom code generation but require specialized programming knowledge. However, FlashTensor can support new models automatically.

8 Concolusion

In this work, to tackle the significant memory overhead problem incurred by long contexts, we propose a DNN optimization system FlashTensor that utilizes fine-grained tensor properties to optimize overall performance. We conclude four key tensor properties, involving reduce dependency, broadcast, size, and value, and identify them from the computation graph. FlashTensor further employs tensor property-aware transformations and kernel mappings to achieve optimal performance. The experiment results reveal that FlashTensor outperforms state-of-the-art works on typical GPU platforms.

Acknowledgments

We would like to thank the anonymous reviewers for their insightful comments. This work is supported by the National Key R&D Program of China under Grant 2023YFB3001501, Beijing Natural Science Foundation (L242017, L243001), NSFC Distinguished Young Scholar (62225206), National Natural Science Foundation of China (U23A6007, 62302251) and Tsinghua University Initiative Scientific Research Program. Jidong Zhai and Yuyang Jin are the corresponding authors of this paper.

References

[1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: KV Cache reduction through key tokens selection for Efficient

Generative Inference. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. Machine Learning and Systems, Santa Clara, California, USA, 114–127. https://proceedings.mlsys.org/paper_files/paper/2024/file/48fecef47b19fe501d27d338b6d52582-Paper-Conference.pdf

[2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. https://doi.org/10.1145/3620665.3640366

[3] Dimitris Bertsimas and John Tsitsiklis. 1993. Simulated annealing. *Statistical science* 8, 1 (1993), 10–15.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, USA, 579–594.

[6] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 3393–3404.

[7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. *cudnn: Efficient primitives for deep learning*. Technical Report. NVIDIA.

[8] NVIDIA cuBLAS. 2016. https://developer.nvidia.com/cublas.

[9] Jincheng Dai, Zhuowei Huang, Haiyun Jiang, Chen Chen, Deng Cai, Wei Bi, and Shuming Shi. 2024. CORM: Cache Optimization with Recent Message for Large Language Model Inference. arXiv:2404.15949 [cs.CL] https://arxiv.org/abs/2404.15949

[10] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] https://arxiv.org/abs/2307.08691

[11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2024. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1189, 16 pages.

[12] FlashInfer. 2024. https://flashinfer.ai/.

[13] Pytorch FlexAttention. 2024. https://pytorch.org/blog/flexattention/.

[14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[15] Muyan Hu, Ashwin Venkatram, Shreyashri Biswas, Balamurugan Marimuthu, Bohan Hou, Gabriele Oliaro, Haojie Wang, Liyan Zheng, Xupeng Miao, Jidong Zhai, and Zhihao Jia. 2024. Optimal Kernel Orchestration for Tensor Programs with Korch. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 755–769. https://doi.org/10.1145/3620666.3651383

[16] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) *(SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 47–62. https://doi.org/10.1145/3341301.3359630

[17] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[18] Chris Lattner, Jacques A. Pienaar, Mehdi Amini, Uday Bondhugula, River Riddle, Albert Cohen, Tatiana Shpeisman, Andy Davis, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A Compiler Infrastructure for the End of Moore's Law. *CoRR* abs/2002.11054 (2020), 21 pages. arXiv:2002.11054 https://arxiv.org/abs/2002.11054

[19] Dengbo Li, Hanning Zhang, Jieren Cheng, and Bernie Liu. 2024. Improving efficiency of DNN-based relocalization module for autonomous driving with server-side computing. *Journal of Cloud Computing* 13, 1 (2024), 25.

[20] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469 [cs.CL] https://arxiv.org/abs/2404.14469

[21] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. DNNFusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) *(PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 883–898. https://doi.org/10.1145/3453483.3454083

[22] Wei Niu, Md Musfiqur Rahman Sanim, Zhihao Shu, Jiexiong Guan, Xipeng Shen, Miao Yin, Gagan Agrawal, and Bin Ren. 2024. SmartMem: Layout Transformation Elimination and Adaptation for Efficient DNN Execution on Mobile. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 916–931. https://doi.org/10.1145/3620666.3651384

[23] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, and others. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, Chapter 721, 12.

[25] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. YaRN: Efficient Context Window Extension of Large Language Models. arXiv:2309.00071 [cs.CL] https://arxiv.org/abs/2309.00071

[26] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. https://openai.com/blog/language-unsupervised. OpenAI Blog.

[27] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[28] Siyu Ren and Kenny Q. Zhu. 2024. On the Efficacy of Eviction Policy for Key-Value Constrained Generative Language Model Inference. arXiv:2402.06262 [cs.CL] https://arxiv.org/abs/2402.06262

[29] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] https://arxiv.org/abs/2308.12950

[30] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. arXiv:2407.08608 [cs.LG] https://arxiv.org/abs/2407.08608

[31] Shahab Shamshirband, Mahdis Fathi, Abdollah Dehzangi, Anthony Theodore Chronopoulos, and Hamid Alinejad-Rokny. 2021. A review on deep learning approaches in healthcare systems: Taxonomies, challenges, and open issues. *Journal of Biomedical Informatics* 113 (2021), 103627.

[32] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 701–718. https://www.usenix.org/conference/osdi23/presentation/shi

[33] Shizhi Tang, Jidong Zhai, Haojie Wang, Lin Jiang, Liyan Zheng, Zhenhao Yuan, and Chen Zhang. 2022. FreeTensor: a free-form DSL with holistic optimizations for irregular tensor programs. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) *(PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 872–887. https://doi.org/10.1145/3519939.3523448

[34] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118 [cs.CL] https://arxiv.org/abs/2408.00118

[35] MosaicML NLP Team. 2023. *Introducing MPT-30B: Raising the bar for open-source foundation models*. MosaicML. www.mosaicml.com/blog/mpt-30b Accessed: 2023-06-22.

[36] NVIDIA TensorRT. 2017. https://developer.nvidia.com/tensorrt.

[37] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. https://doi.org/10.1145/3315508.3329973

[38] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] https://arxiv.org/abs/2302.13971

[39] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc., Long Beach, California, USA. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[41] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. 2021. PET: Optimizing Tensor Programs with Partially Equivalent Transformations and Automated Corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, Virtual, 37–54. https://www.usenix.org/conference/osdi21/presentation/wang

[42] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[43] TensorFlow XLA. 2023. https://www.tensorflow.org/xla.

[44] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞Bench: Extending Long Context Evaluation Beyond 100K Tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15262–15277. https://doi.org/10.18653/v1/2024.acl-long.814

[45] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2024. H2O: heavy-hitter oracle for efficient generative inference of large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1506, 50 pages.

[46] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 49, 17 pages.

[47] Liyan Zheng, Haojie Wang, Jidong Zhai, Muyan Hu, Zixuan Ma, Tuowei Wang, Shuhong Huang, Xupeng Miao, Shizhi Tang, Kezhao Huang, and Zhihao Jia. 2023. EINNET: Optimizing Tensor Programs with Derivation-Based Transformations. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 739–755. https://www.usenix.org/conference/osdi23/presentation/zheng

[48] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, Shuaiwen Leon Song, and Wei Lin. 2022. AStitch: Enabling a New Multi-Dimensional Optimization Space for Memory-Intensive ML Training and Inference on Modern SIMT Architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 359–373. https://doi.org/10.1145/3503222.3507723

A Appendix

FlashTensor is an open-source project, publicly accessible at https://github.com/monellz/FlashTensor. Instructions for reproducing the results reported in the paper are available at https://github.com/monellz/FlashTensor-AE and https://zenodo.org/records/14220175.